

■ AI Interview Evaluator

Project Documentation
System Design Final Project

Powered by Google Gemini 1.5 Flash · Streamlit · SQLite

Project Title	AI-Based Interview Evaluator
Technology	Python · Streamlit · Google Gemini AI
Database	SQLite via SQLAlchemy 2.0
Submission Date	June 2026

Table of Contents

1.	Problem Statement	3
2.	Proposed Solution	4
3.	System Architecture	5
4.	Module Description	6
5.	Database Design	8
6.	Technology Stack	9
7.	Implementation Details	10
8.	Evaluation Rubric & Scoring	12
9.	Screenshots	13
10.	Future Scope	14

1. Problem Statement

The modern hiring process relies heavily on in-person or remote technical interviews to assess a candidate's knowledge, communication skills, and problem-solving abilities. However, candidates often lack access to structured, objective, and domain-specific practice tools. Existing resources are either too generic (basic quiz apps), too expensive (coaching services), or provide no actionable feedback beyond a simple right/wrong score.

Core Challenges Identified

- **Lack of structured practice environments** — candidates cannot simulate real interview pressure with immediate, personalised feedback.
- **Subjectivity in self-assessment** — without expert evaluation, candidates cannot identify blind spots in their knowledge.
- **Domain-specific gaps** — different interview types (system design, coding, behavioural) require different preparation approaches, rarely addressed by a single tool.
- **No progress tracking** — most practice tools do not maintain a history of performance, making it impossible to measure improvement over time.
- **Accessibility** — professional coaching or mock interview services can be prohibitively expensive for students and early-career professionals.

Problem Definition: Design and implement a scalable, AI-powered web application that provides domain-specific mock interviews with automated, rubric-based evaluation, detailed performance analytics, progress tracking, and exportable reports — accessible to any user with a web browser.

2. Proposed Solution

The AI Interview Evaluator is a full-stack Python web application built with Streamlit as the presentation layer and Google Gemini 1.5 Flash as the evaluation engine. It provides a complete interview simulation pipeline from question selection to performance reporting.

Solution Highlights

- **225-question bank** spanning 5 domains (Software Engineering, System Design, Data Science & ML, HR/Behavioural, Product Management) at 3 difficulty levels (Easy, Medium, Hard).
- **AI-powered per-question evaluation** using a structured JSON prompt sent to Gemini 1.5 Flash, which returns scores on 4 weighted rubric criteria, strengths, areas for improvement, and an ideal answer hint.
- **Rubric-based weighted scoring** — each domain has unique weights (e.g., Software Engineering: Technical Accuracy 35%, Depth 25%, Communication 25%, Problem-Solving 15%) ensuring fair, objective scoring.
- **SQLite persistence** via SQLAlchemy 2.0 ORM — all candidates, interviews, and per-question evaluations are stored in a relational database with cascade deletes.
- **Rich analytics dashboard** using Plotly — score trend lines, radar charts, grade distribution histograms, and domain-wise performance comparisons.
- **PDF and CSV export** via ReportLab and pandas — multi-page professional reports with cover page, performance summary, Q&A analysis, and domain recommendations.
- **Global leaderboard** with privacy masking (only first 3 characters of names shown) ranked by best percentage score per domain.
- **Admin dashboard** for platform-wide analytics, candidate management, and data export.
- **Offline/mock mode** — setting `USE MOCK=true` runs the entire pipeline without any API calls, producing realistic scores (4–9) for demonstration and testing.

Design Principles Followed

- **Separation of Concerns** — UI, business logic, and data access are cleanly separated into distinct Python packages.
- **DRY (Don't Repeat Yourself)** — a single `_build_result()` function in `evaluator.py` produces evaluation dicts consumed by all UI pages.
- **Backward Compatibility** — all legacy key names are aliased so no existing functionality breaks when internal keys are refactored.
- **Idempotent Initialisation** — `init_db()` is safe to call on every Streamlit rerun; tables are created only if they don't exist.
- **Graceful Degradation** — if the Gemini API call fails after 3 retries, the system automatically falls back to mock evaluation and flags the result.

3. System Architecture

The system follows a layered architecture with clear boundaries between presentation, business logic, data access, and external services. Streamlit's session state acts as an in-memory state machine that coordinates the interview flow across page reruns.

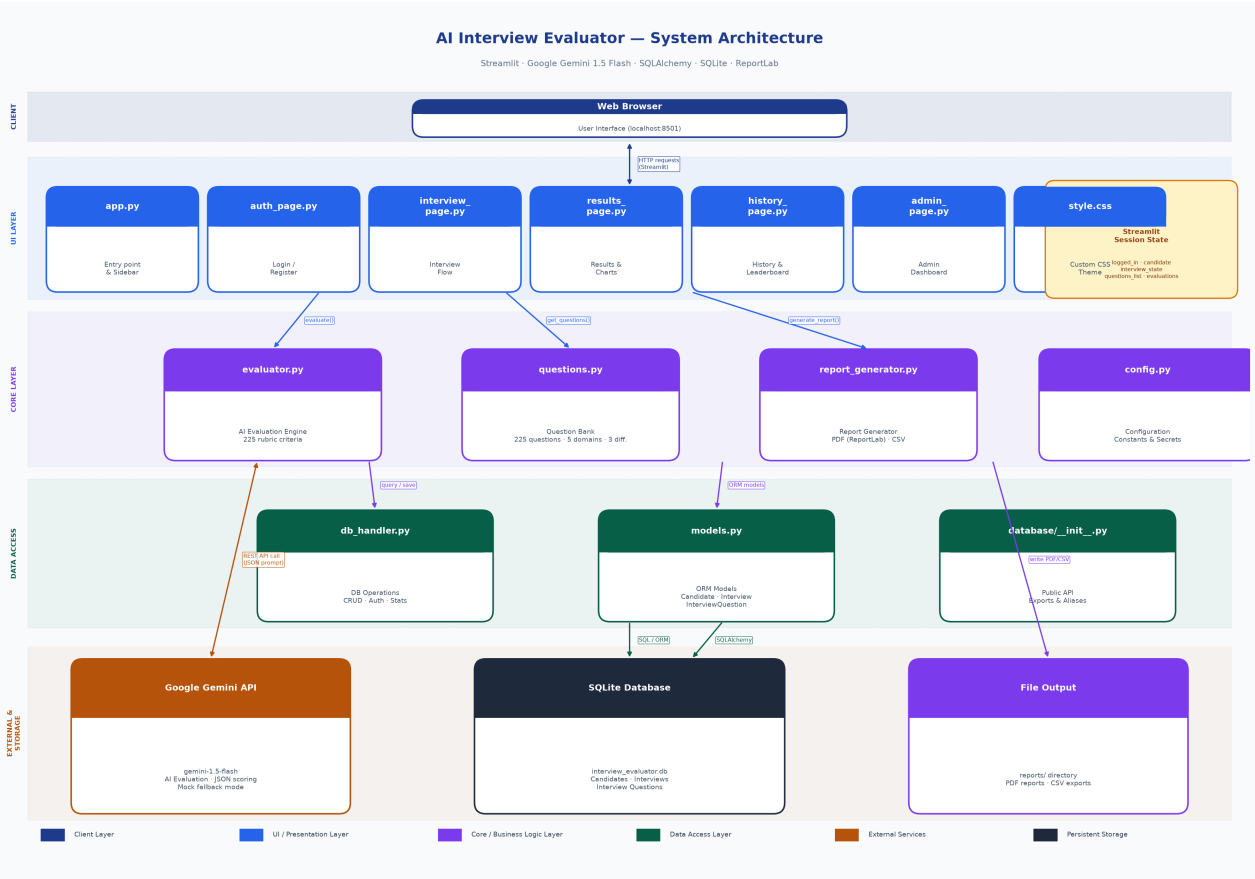


Figure 1: System Architecture Diagram — 6 layers from browser client to SQLite storage

Layer Description

Layer	Components	Responsibility
Client Layer	Web Browser	User interface via HTTP on port 8501
UI / Presentation	app.py, auth_page, interview_page, results_page, history_page, admin_page	Page rendering, navigation, session state management
Core / Logic	evaluator.py, questions.py, report_generator.py, config.py	AI evaluation, question retrieval, report generation
Data Access	db_handler.py, models.py, database/_init_.py	ORM queries, CRUD operations, stats aggregation
External Services	Google Gemini 1.5 Flash API	AI scoring, rubric evaluation, feedback generation
Persistent Storage	SQLite (interview_evaluator.db)	Candidate accounts, interview records, question scores

Interview State Machine

The interview flow is implemented as a state machine managed through Streamlit session state:

not_started → (Begin button) → **in_progress** → (Submit / Skip) → **evaluating** → (AI call complete)
→ back to **in_progress** (next Q) or → **completed** → (save to DB) → Results page

4. Module Description

4.1 app.py — Application Entry Point

The root Streamlit file. Responsibilities: load .env, call `init_db()` and `init_session_state()` on every run, inject custom CSS, render the sidebar navigation, and route to the appropriate page function based on `current_page` session key.

4.2 config.py — Configuration Constants

Centralised configuration: `DOMAINS` (5), `DIFFICULTY_LEVELS` (3), `QUESTION_COUNT_OPTIONS` ([5,7,10]), `DOMAIN_ICONS`, `DIFFICULTY_COLORS`, `GRADE_COLORS`, `GEMINI_MODEL`, `DB_PATH`, `MAX_ANSWER_WORDS`, `EVALUATION_TIMEOUT`.

4.3 core/evaluator.py — AI Evaluation Engine

Core module containing the Gemini integration. Key functions:

- `evaluate_answer(question, answer, domain, difficulty, expected_keywords)` — builds a structured prompt, calls Gemini with 3-attempt retry + exponential backoff, parses JSON, validates and recomputes weighted score from rubric.
- `mock_evaluate()` — returns realistic scores (4–9) without API calls. Used when `USE MOCK=true` or as automatic fallback after API failure.
- `get_interview_summary_feedback()` — calls Gemini to generate an AI narrative summary of the full interview.
- `RUBRICS dict` — 5 domains × 4 criteria with weights summing to 100. Used both for prompt construction and weighted score recomputation.

4.4 core/questions.py — Question Bank

Stores all 225 interview questions as a Python dictionary. Structure: `QUESTIONS[domain]` = list of dicts with fields: `id`, `question`, `difficulty`, `topic`, `expected_keywords`, `follow_up`. The `get_questions(domain, count, difficulty)` function filters and randomly samples the requested number of questions.

4.5 core/report_generator.py — Report Generator

Generates professional output using ReportLab Platypus. Two public functions:

- `generate_pdf_report(interview_data, candidate_data, evaluations=None)` → bytes — 4-page PDF: Cover, Performance Summary, Question Analysis (with matplotlib bar chart), Recommendations.
- `generate_csv_report(interview_data, evaluations=None)` → bytes — 12-column CSV: Q#, Topic, Domain, Difficulty, Question, Answer, Score, Grade, Strengths, Improvements, Ideal Answer Hint, Time Taken.

4.6 database/models.py — ORM Models

Defines 3 SQLAlchemy models with cascade relationships:

- **Candidate** — `id`, `name`, `email`, `password_hash` (salt:sha256), `created_at`, `is_admin`.
- **Interview** — `id`, `candidate_id` (FK), `domain`, `difficulty`, `total_questions`, `overall_score`, `percentage`, `letter_grade`, `verdict`, `started_at`, `completed_at`, `duration_minutes`, `status`.

- **InterviewQuestion** — id, interview_id (FK), question_number, question_text, difficulty, topic, candidate_answer, score, grade, strengths, improvements, ideal_answer_hint, time_taken_seconds.

4.7 database/db_handler.py — Data Access Layer

All database operations in a single module. Uses a @contextmanager _session() that auto-commits on success and rolls back on exception. Key functions: init_db (idempotent, seeds admin + demo user), register_candidate, login_candidate (salted SHA-256 verify), save_interview, get_candidate_interviews, get_interview_details, get_leaderboard, get_domain_statistics, get_recent_activity, plus legacy aliases for backward compatibility.

4.8 – 4.12 UI Pages

File	Purpose	Key Features
auth_page.py	Authentication	Login / Register forms, init_session_state() with 18 keys, auto-login after register
interview_page.py	Interview Flow	State machine, per-question answer input, word count indicator, AI evaluation sp
results_page.py	Results Display	Score hero, metric cards, Plotly charts (radar, bar, pie), Q&A breakdown, AI narr
history_page.py	History + Leaderboard	Interview cards, domain/grade/date filters, trend chart, privacy-masked leaderbo
admin_page.py	Admin Dashboard	Platform stats, domain analytics, candidates table, activity feed, score histogram

5. Database Design

The application uses SQLite as its persistent store, accessed through SQLAlchemy 2.0 ORM. The schema consists of 3 tables with a one-to-many cascade delete relationship chain.

Entity-Relationship Overview

Candidate (1) ■■■■ (∞) Interview (1) ■■■■ (∞) InterviewQuestion

On Candidate delete → cascade delete all Interviews → cascade delete all InterviewQuestions

Table: candidates

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique candidate identifier
name	VARCHAR(100)	NOT NULL	Full display name
email	VARCHAR(150)	UNIQUE, NOT NULL	Login email address
password_hash	VARCHAR(256)	NOT NULL	'salt:sha256hex' format
created_at	DATETIME	DEFAULT utcnow	Account creation timestamp
is_admin	BOOLEAN	DEFAULT False	Admin privilege flag

Table: interviews

Column	Type	Description
id	INTEGER PK	Auto-incremented interview ID
candidate_id	INTEGER FK	References candidates.id (cascade delete)
domain	VARCHAR(100)	Interview domain (e.g. Software Engineering)
difficulty	VARCHAR(50)	Easy / Medium / Hard
total_questions	INTEGER	Number of questions answered
overall_score	FLOAT	Sum of individual question scores
percentage	FLOAT	overall_score / (total_questions × 10) × 100
letter_grade	VARCHAR(5)	A+ / A / B+ / B / C+ / C / D / F
verdict	VARCHAR(200)	AI-generated performance verdict text
started_at	DATETIME	Interview start timestamp (UTC)
completed_at	DATETIME	Interview end timestamp (UTC)
duration_minutes	FLOAT	Computed from start/end timestamps
status	VARCHAR(20)	Always 'completed' for saved interviews

Table: interview_questions

Column	Type	Description
id	INTEGER PK	Auto-incremented row ID
interview_id	INTEGER FK	References interviews.id (cascade delete)
question_number	INTEGER	Position within the interview (1-based)
question_text	TEXT	Full question string
difficulty	VARCHAR(50)	Per-question difficulty level
topic	VARCHAR(100)	Sub-topic tag (e.g. OOP, Recursion)
candidate_answer	TEXT	Candidate's submitted answer text
score	INTEGER	AI score 1–10
grade	VARCHAR(50)	Excellent / Good / Average / Poor
strengths	TEXT	AI-identified answer strengths
improvements	TEXT	AI-suggested improvements
ideal_answer_hint	TEXT	Model answer hint from AI
time_taken_seconds	INTEGER	Seconds spent on this question

6. Technology Stack

Layer	Technology	Version	Role
Web Framework	Streamlit	≥ 1.32	Single-page app routing, session state, widget rendering
AI Engine	Google Gemini 1.5 Flash	via google-generativeai	Rubric-based answer evaluation, narrative summaries
Programming Language	Python	3.10+	All backend logic, data processing, scripting
Database ORM	SQLAlchemy	≥ 2.0	Model definition, query building, session management
Database	SQLite	Built-in	Lightweight relational storage, zero-configuration
Charting	Plotly	≥ 5.20	Interactive line, bar, radar, pie charts in UI
Diagram / Charts	Matplotlib	≥ 3.8	Static bar chart in PDF, architecture diagram PNG
PDF Generation	ReportLab (Platypus)	≥ 4.1	Multi-page professional PDF with custom templates
Tabular Data	pandas	≥ 2.2	DataFrame operations, CSV export
Numerical	NumPy	≥ 1.26	Stats computations in admin score histogram
Environment	python-dotenv	≥ 1.0	Loads GEMINI_API_KEY and USE MOCK from .env file
Styling	Custom CSS	—	Inter font, CSS variables, animations via style.css

7. Implementation Details

7.1 AI Evaluation Pipeline

Each submitted answer goes through the following pipeline:

- **Step 1 — Prompt Construction:** evaluator.py builds a structured prompt including the question, candidate answer, domain, difficulty, expected keywords, and the domain rubric with criteria weights.
- **Step 2 — API Call with Retry:** The prompt is sent to Gemini 1.5 Flash with temperature=0.3 and max_output_tokens=1024. If the call fails, it retries up to 3 times with exponential backoff (1s, 2s, 4s).
- **Step 3 — JSON Parsing:** Gemini returns a JSON object. The parser extracts scores for each rubric criterion, validates them, and recomputes the weighted score using our rubric weights (not Gemini's calculation).
- **Step 4 — Result Building:** A standardised result dict is constructed with new keys (weighted_score, grade, strengths, improvements, ideal_answer_hint, follow_up_question, keyword_coverage) plus backward-compatible aliases.
- **Step 5 — Fallback:** If all 3 API attempts fail, mock_evaluate() is called and result['_fallback'] = True is set. A warning is shown to the user.

7.2 Password Security

Passwords are never stored in plaintext. The _hash_password() function generates a cryptographically random 16-byte salt using secrets.token_hex(16), then computes SHA-256(salt + password) and stores the result as 'salt:hexdigest'. Verification recomputes and compares using hmac-like comparison.

7.3 Session State Management

Streamlit re-executes the entire script on every user interaction. The init_session_state() function in auth_page.py initialises 18 session state keys with safe defaults on the first run, and is idempotent on subsequent runs. The interview state machine uses the 'interview_state' key as its current state.

7.4 Mock Mode Architecture

When USE MOCK=true in .env, _is_mock_mode() returns True and evaluate_answer() delegates directly to mock_evaluate() without any API call. The mock evaluator:

- Computes a base score from answer length (4.5 for short, 5.5 for medium, 7.0 for longer, 8.0 for detailed answers).
- Applies keyword bonus (+1 point maximum for keyword coverage).
- Applies difficulty adjustment (± 0.5).
- Clamps the final weighted score to [4.0, 9.0] for real answers and [1.0, 3.0] for skipped answers, ensuring realistic demo scores.
- Returns identical structure to the real evaluator — zero code changes needed elsewhere.

7.5 Report Generation

The generate_pdf_report() function uses ReportLab's Platypus framework with two PageTemplate objects (Cover and Content). Each page has a custom background drawn via canvas.saveState()/restoreState(). The report includes:

- **Page 1 — Cover:** Navy gradient background, candidate name, domain, difficulty, grade, overall score, date.

- **Page 2 — Performance Summary:** Score metrics table, matplotlib bar chart (embedded as RImage), verdict box.
- **Pages 3+ — Question Analysis:** Each question wrapped in KeepTogether([...]) with question box, answer box, score banner, strengths/improvements panels.
- **Last Page — Recommendations:** Domain-specific study tips from the _DOMAIN_TIPS dict.

7.6 Leaderboard Privacy

The leaderboard masks all candidate names except the current user's own name. The `_mask_name()` function takes the first word of the name, keeps the first 3 characters visible, and appends '*****'. Example: 'Kishan Patel' → 'Kis*****'. The current user's row is highlighted in light blue with a 'You' badge.

7.7 Database Seeding

On every application startup, `init_db()` calls two seed functions: `_seed_admin()` creates `admin@ai.com/admin123` with `is_admin=True` if it doesn't exist, and `_seed_demo_user()` creates `demo@test.com/demo123` if it doesn't exist. Both functions are wrapped in `try/except` and log warnings on failure without crashing the app.

8. Evaluation Rubric & Scoring

Each domain has a unique rubric with 4 criteria and weights summing to 100. The AI evaluates each criterion independently, and the final weighted score is computed deterministically from the rubric — never trusted directly from the AI.

Domain Rubrics

Domain	Criterion 1	Criterion 2	Criterion 3	Criterion 4
Software Engineering	Technical Accuracy (35%)	Depth & Completeness (25%)	Communication Clarity (25%)	Problem-Solving Approach (15%)
System Design	Scalability (30%)	Architecture Depth (30%)	Trade-off Analysis (25%)	Communication Clarity (15%)
Data Science & ML	Conceptual Accuracy (35%)	Mathematical Depth (25%)	Practical Application (25%)	Communication (15%)
HR / Behavioral	Situation Clarity (25%)	Action & Reasoning (35%)	Outcome & Impact (25%)	Communication (15%)
Product Management	Problem Definition (25%)	Strategic Thinking (30%)	Data-Driven Approach (25%)	Communication (20%)

Letter Grade Scale

Percentage	Letter Grade	Interpretation
≥ 90%	A+	Outstanding — Expert level, highly recommended
≥ 80%	A	Excellent — Strong competency demonstrated
≥ 72%	B+	Good — Above average, minor gaps
≥ 64%	B	Good — Solid foundation with some areas to improve
≥ 56%	C+	Satisfactory — Core concepts understood
≥ 48%	C	Average — Acceptable but needs focused preparation
≥ 35%	D	Below Average — Significant gaps identified
< 35%	F	Fail — Fundamental review required

9. Screenshots

The following screens represent the primary user interface views of the application. Run the application locally and navigate through each page to see the live UI.

Screen	Description	How to Access
Login / Register	Auth page with two tabs, demo credentials box, gradient card	Open app — shown before login
Home Dashboard	Welcome banner, 4 metric cards, Quick Start panel, trend line chart	Login → Home (sidebar)
Interview Setup	Domain/difficulty/count config, live preview card, rules panel	Start Interview → Setup tab
Interview Q&A	Progress bar, question card, text area, word count indicator	During an active interview session
Evaluation Result	Score hero, grade badge, strengths panel, improvements panel	After submitting an answer
Results Page	Score hero, radar chart, bar chart, Q&A breakdown table	After completing interview
PDF Report	Professional multi-page PDF with cover, charts, Q&A analysis	Results → Download PDF
Interview History	Filtered interview cards, trend line chart across sessions	Sidebar → Interview History
Leaderboard	Ranked table, masked names, You badge, domain filter	History → Leaderboard tab
Admin Dashboard	Platform stats, domain analytics, candidates table, danger zone	Login as admin → Admin Panel

■ To capture screenshots for your submission: launch the app with `streamlit run app.py`, navigate to each page, and use your OS screenshot tool (macOS: `Cmd+Shift+4`, Windows: `Win+Shift+S`). Insert the captured images here before final PDF submission.

10. Future Scope

The current implementation provides a strong foundation for a production-grade interview platform. The following enhancements are planned or proposed for future versions:

10.1 AI & Evaluation Enhancements

- **Voice Input:** Integrate Web Speech API or Whisper to allow candidates to speak answers rather than type, mimicking real interview conditions.
- **Code Execution Environment:** Embed a sandboxed Python/JavaScript REPL for coding questions, evaluating both correctness and code quality.
- **Adaptive Difficulty:** Dynamically adjust question difficulty based on the candidate's running score — increase difficulty after correct answers, decrease after poor ones.
- **Multi-Model Support:** Allow users to choose between Gemini, GPT-4, and Claude for evaluation, comparing scoring consistency across models.

10.2 Platform Features

- **Interview Scheduling:** Calendar integration allowing candidates to schedule mock interviews and receive email/SMS reminders.
- **Peer Review Mode:** Allow two users to evaluate each other's answers with a crowd-sourced score alongside the AI score.
- **Company-Specific Question Banks:** Curated question sets modeled after FAANG, startup, and domain-specific company interviews.
- **Interview Streaks & Gamification:** Daily practice streaks, badges, XP points, and level progression to improve engagement.

10.3 Technical Improvements

- **PostgreSQL Migration:** Replace SQLite with PostgreSQL for multi-user concurrent access in production deployments.
- **Redis Caching:** Cache leaderboard queries and domain statistics with a 5-minute TTL to reduce DB load.
- **REST API Layer:** Expose a FastAPI backend so the evaluation engine can serve mobile apps and third-party integrations.
- **Docker + Kubernetes:** Containerise the application for one-command deployment and horizontal scaling.
- **Automated Testing Suite:** Add pytest unit tests for evaluator.py, db_handler.py, and report_generator.py with CI/CD integration.
- **OAuth 2.0 Authentication:** Replace email/password login with Google Sign-In and GitHub OAuth for simpler onboarding.

10.4 Analytics & Reporting

- **Comparative Analytics:** Compare a candidate's scores against the anonymised cohort average for each domain and difficulty.
- **AI-Generated Study Plan:** After each interview, Gemini generates a personalised 2-week study plan targeting identified weak areas.
- **Employer Portal:** Allow companies to create accounts, set custom question banks, send interview links to candidates, and view scores in a shared dashboard.

Conclusion: The AI Interview Evaluator successfully demonstrates the integration of modern AI capabilities (Google Gemini) with a full-stack Python web application (Streamlit + SQLAlchemy). It addresses a real-world problem in interview preparation by providing objective, rubric-based feedback at scale, with a robust architecture that supports future enhancements without requiring a ground-up rewrite.